



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERU
FACULTAD DE INGENIERIA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Práctica SEMANA 12

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad II: Desarrollo Web FullStack

Tema: Gestión de Formularios, Django Admin, Middleware, Sesiones, Autenticación y Autorización

Modalidad: Presencial / Laboratorio

Fecha: 24.06.2026

APELLIDOS Y NOMBRES: Barja Ortiz Erick Gerson

OBJETIVO DE LA PRÁCTICA

Dominar el ciclo completo de gestión de datos y seguridad en Django 5.x mediante la implementación de formularios robustos (clásicos y ModelForm), validación en múltiples etapas (clean(), clean_(), validadores personalizados), sanitización de entrada, integración con plantillas y manejo de errores estructurados. Configurar y personalizar el Django Admin para administración operativa (registro, filtros, búsquedas, campos calculados, acciones masivas). Comprender e implementar el ciclo de Middleware (interceptación request/response), gestión de sesiones seguras, autenticación (login/logout/@login_required), autorización basada en roles/permisos, y protección CSRF. Integrar todos los componentes en un sistema backend funcional, auditado y alineado a estándares de seguridad web.

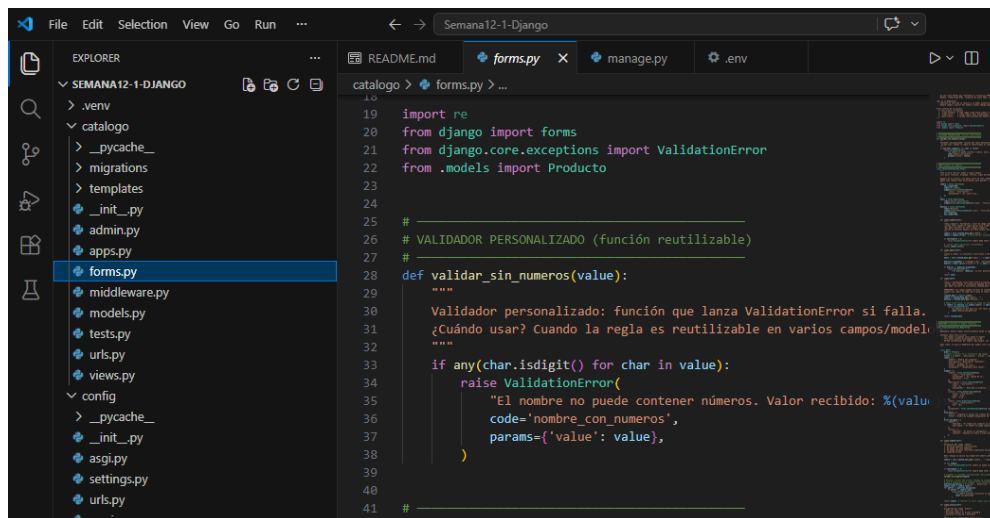
RECURSOS REQUERIDOS

- Python 3.11+ + Django 5.x + pip
- Visual Studio Code (Extensiones: Python, Django, Pylint, SQLite Viewer, REST Client)
- Terminal integrada + entorno virtual (venv)
- Herramientas: django-extensions + shell_plus, Postman/Insomnia (pruebas auth), DevTools (Cookies/Storage, Network)
- Documentación oficial: docs.djangoproject.com/en/5.0/topics/forms/, [admin/](https://docs.djangoproject.com/en/5.0/topics/admin/), [middleware/](https://docs.djangoproject.com/en/5.0/topics/middleware/), [auth/](https://docs.djangoproject.com/en/5.0/topics/auth/)

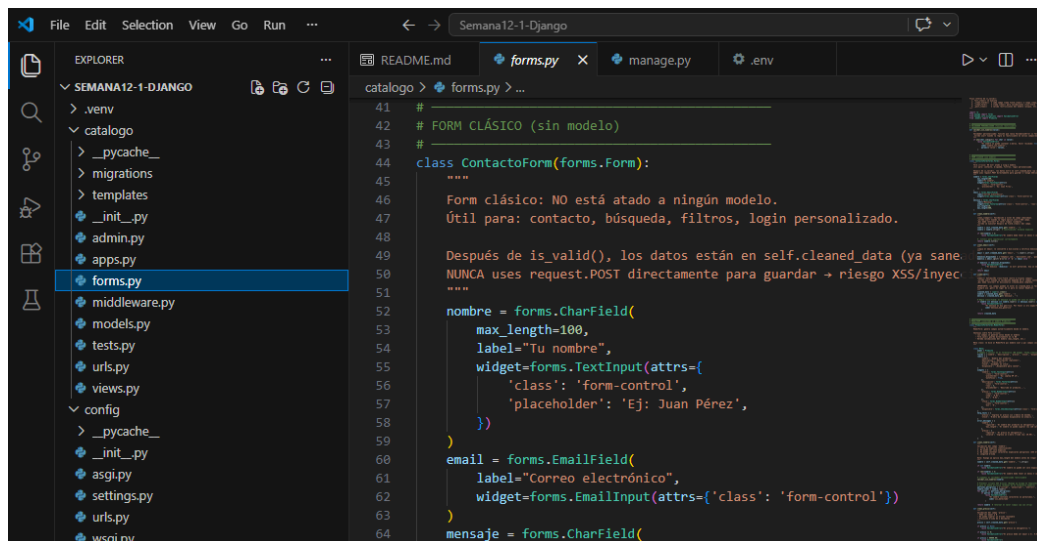
- GitHub (control de versiones)

EJERCICIO PASO A PASO

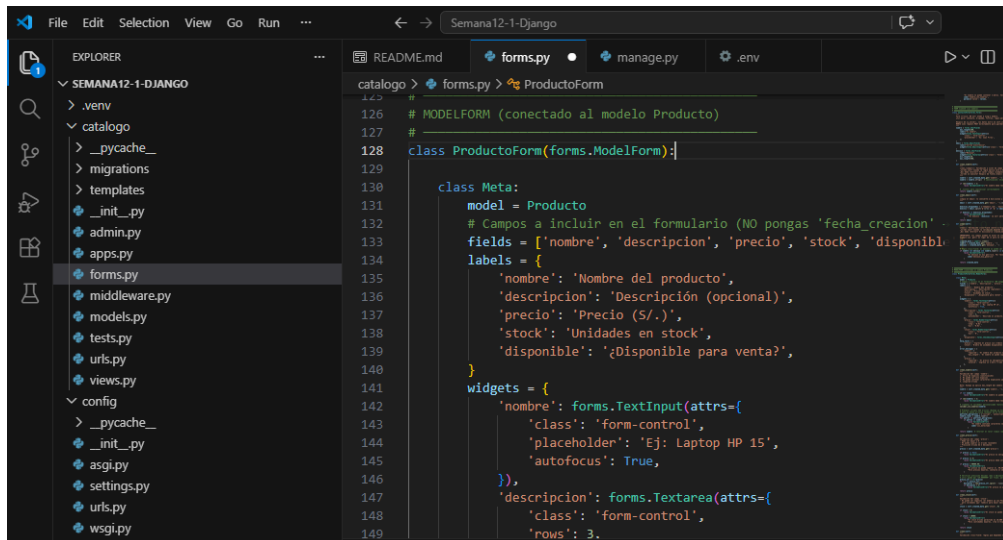
1. Formularios & Plantillas: Crear forms.py, implementar ModelForm y formulario clásico. Renderizar en template con `{{ form.as_p }}`, `{{ form.errors }}`, `csrf_token`. Configurar POST route.



```
19 import re
20 from django import forms
21 from django.core.exceptions import ValidationError
22 from .models import Producto
23
24
25 #
26 # VALIDADOR PERSONALIZADO (función reutilizable)
27 #
28 def validar_sin_numeros(value):
29     """
30     Validador personalizado: función que lanza ValidationError si falla.
31     ¿Cuándo usar? Cuando la regla es reutilizable en varios campos/modelos.
32     """
33     if any(char.isdigit() for char in value):
34         raise ValidationError(
35             "El nombre no puede contener números. Valor recibido: %(value)s",
36             code='nombre_con_numeros',
37             params={'value': value},
38         )
39
40 #
```



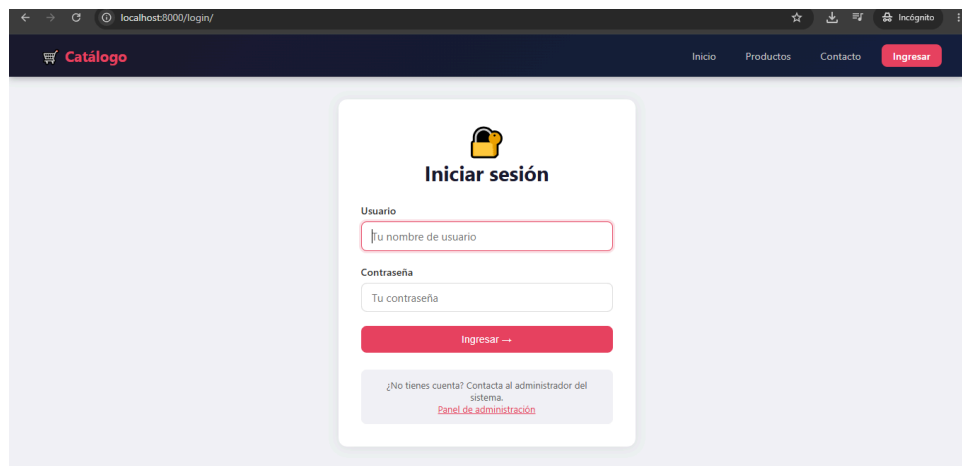
```
41 #
42 # FORM CLÁSICO (sin modelo)
43 #
44 class ContactoForm(forms.Form):
45     """
46     Form clásico: NO está atado a ningún modelo.
47     Útil para: contacto, búsqueda, filtros, login personalizado.
48
49     Después de is_valid(), los datos están en self.cleaned_data (ya saneado).
50     NUNCA uses request.POST directamente para guardar → riesgo XSS/inyección.
51     """
52     nombre = forms.CharField(
53         max_length=100,
54         label="Tu nombre",
55         widget=forms.TextInput(attrs={
56             'class': 'form-control',
57             'placeholder': 'Ej: Juan Pérez',
58         })
59     )
60     email = forms.EmailField(
61         label="Correo electrónico",
62         widget=forms.EmailInput(attrs={'class': 'form-control'})
63     )
64     mensaje = forms.CharField(
```



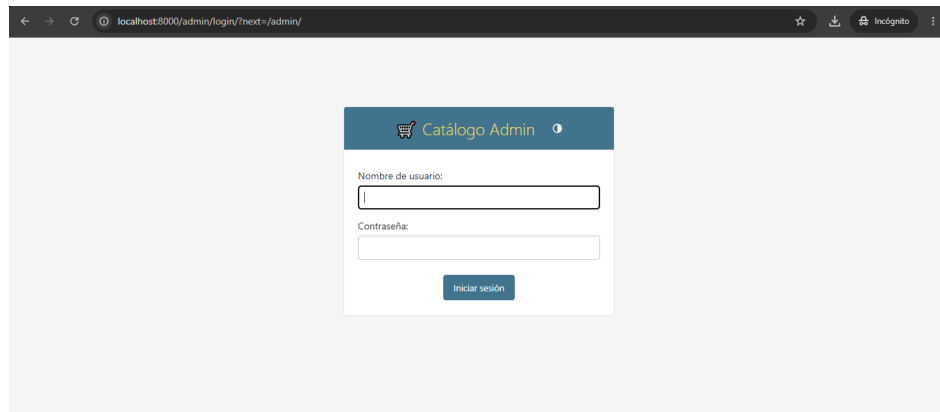
The image shows a VS Code editor window with a Django project named 'SEMANA12-1-DJANGO'. The file explorer on the left shows the project structure, including files like 'forms.py', 'models.py', 'views.py', and 'urls.py'. The main editor window displays the 'forms.py' file, which contains a Django form class named 'ProductoForm'. The code defines the model fields to be included in the form, their labels, and the corresponding Django widgets. The form is connected to the 'Producto' model.

```
126 # MODELFORM (conectado al modelo Producto)
127 #
128 class ProductoForm(forms.ModelForm):
129
130     class Meta:
131         model = Producto
132         # Campos a incluir en el formulario (NO pongas 'fecha_creacion'
133         fields = ['nombre', 'descripcion', 'precio', 'stock', 'disponibl
134         labels = {
135             'nombre': 'Nombre del producto',
136             'descripcion': 'Descripción (opcional)',
137             'precio': 'Precio ($/.)',
138             'stock': 'Unidades en stock',
139             'disponible': '¿Disponible para venta?',
140         }
141         widgets = {
142             'nombre': forms.TextInput(attrs={
143                 'class': 'form-control',
144                 'placeholder': 'Ej: Laptop HP 15',
145                 'autofocus': True,
146             }),
147             'descripcion': forms.Textarea(attrs={
148                 'class': 'form-control',
149                 'rows': 3,
```

- Validación & Sanitización: Implementar `clean_()`, `clean()` cross-field, validadores regex, `sanitize_input()` básico. Mostrar errores por campo vs. `__all__`.



- Django Admin: Registrar modelos, personalizar `list_display`, `list_filter`, `search_fields`, `readonly_fields`, actions. Crear campo calculado (`@admin.display`).



```

File Edit Selection View Go Run ... Semana12-1-Django
EXPLORER
SEMANA12-1-DJANGO
  catalogo
    migrations
    _pycache_
    _init_.py
    0001_initial.py
  templates\catalogo
    base.html
    catalogo.html
    confirmar_eliminar.html
    contacto.html
    crear_producto.html
    dashboard.html
    home.html
    login.html
    logout_confirm.html
    _init_.py
    admin.py
    apps.py
    forms.py
    middleware.py
    models.py
  README.md
  forms.py
  catalogo.html
  middleware.py
  admin.py
  catalogo > admin.py > marcar_disponible
24 # ACCIÓN PERSONALIZADA
25 #
26
27 def marcar_disponible(modeladmin, request, queryset):
28     actualizados = queryset.update(disponible=True)
29     modeladmin.message_user(
30         request,
31         f'✅ {actualizados} producto(s) marcado(s) como disponibles.',
32         level='success'
33     )
34
35     marcar_disponible.short_description = "✅ Marcar como disponibles"
36
37
38 def marcar_no_disponible(modeladmin, request, queryset):
39     """Acción masiva: desactivar productos."""
40     actualizados = queryset.update(disponible=False)
41     modeladmin.message_user(
42         request,
43         f'❌ {actualizados} producto(s) marcado(s) como no disponibles.',
44         level='warning'
45     )
46
47

```

4. CMiddleware & Sesiones: Crear middleware custom (`__init__`, `__call__`). Configurar `SESSION_ENGINE`, `SESSION_COOKIE_AGE`, `CSRF_COOKIE_SECURE`. Implementar `@login_required`, `PermissionRequiredMixin`.

```

File Edit Selection View Go Run ... Semana12-1-Django
EXPLORER
SEMANA12-1-DJANGO
  catalogo
    migrations
    _pycache_
    _init_.py
    0001_initial.py
  templates\catalogo
    base.html
    catalogo.html
    confirmar_eliminar.html
    contacto.html
    crear_producto.html
    dashboard.html
    home.html
    login.html
    logout_confirm.html
    _init_.py
    admin.py
    apps.py
    forms.py
    middleware.py
    models.py
  README.md
  forms.py
  catalogo.html
  middleware.py
  admin.py
  catalogo > middleware.py > AuditoriaMiddleware > _call_
20
21 import logging
22 import time
23 from django.utils.deprecation import MiddlewareMixin
24
25 # Logger específico para auditoría (configurado en settings.py)
26 logger = logging.getLogger('catalogo.audit')
27
28
29 class AuditoriaMiddleware:
30
31     def __init__(self, get_response):
32         self.get_response = get_response
33         logger.info("AuditoriaMiddleware inicializado.")
34
35     def __call__(self, request):
36         # — PROCESAMIENTO DEL REQUEST (antes de la vista) —
37         inicio = time.time()
38
39         # Obtener IP real (considera proxies/load balancers)
40         ip_cliente = self.obtener_ip(request)
41
42         # Guardar última URL visitada en sesión (útil para redirect-afte
43         # request.session es un diccionario especial que persiste entre

```

```

class AuditoriaMiddleware:
    def __init__(self, request):
        return request.META.get('REMOTE_ADDR', '0.0.0.0')

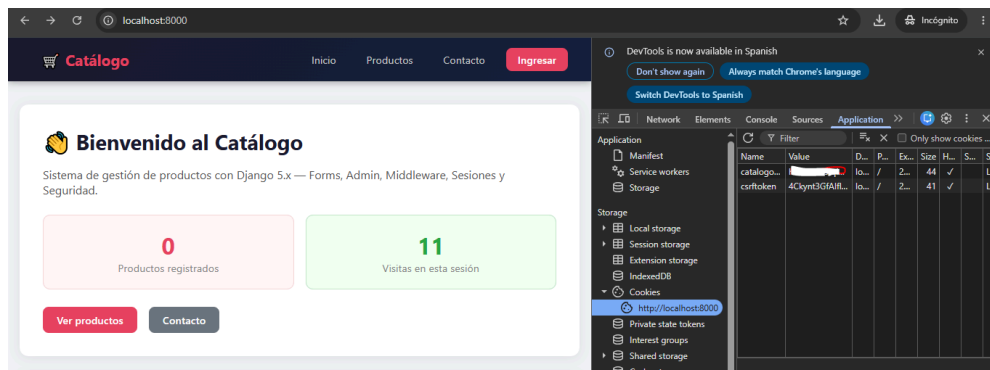
class SessionInfoMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Contar visitas del usuario en esta sesión
        visitas = request.session.get('total_visitas', 0)
        request.session['total_visitas'] = visitas + 1

        # Primera visita: registrar timestamp
        if visitas == 0:
            from django.utils import timezone
            request.session['primera_visita'] = timezone.now().isoformat

        response = self.get_response(request)
        return response
  
```

5. Integración & Auditoría: Probar flujo completo: registro → login → acceso restringido → admin → logout. Verificar cookies, headers XFrame-Options, ContentSecurity-Policy. Logs de middleware.



```

[2026-06-24 13:56:55] INFO catalogo.audit - [POST] /admin/login/ | Status: 200 | Usuario: anónimo
| IP: 127.0.0.1 | Tiempo: 1251.3ms
[24/Jun/2026 13:56:55] "POST /admin/login/?next=/admin/ HTTP/1.1" 200 4410
[2026-06-24 13:59:07] WARNING catalogo.audit - [GET] /.well-known/appspecific/com.chrome.devtools
.json | Status: 404 | Usuario: anónimo | IP: 127.0.0.1 | Tiempo: 19.6ms
Not Found: /.well-known/appspecific/com.chrome.devtools.json
  
```

Link en GitHub: <https://github.com/GErickOBTS/Semana12-1-Django>